

REPORT

Title

Implementation of the N-Queens Problem Using Python

Aim

To implement the N-Queens Problem in Python and determine a valid arrangement of N queens on an $N \times N$ chessboard such that no two queens attack each other.

Problem Statement

The N-Queens Problem is a constraint satisfaction problem in which N queens are placed on an $N \times N$ chessboard. The objective is to position all queens so that no two queens share the same row, column, or diagonal. The program should determine and display a valid arrangement of queens that satisfies all the given constraints.

Requirements

- Python 3.x

Procedure

1. Represent the chessboard using a one-dimensional array.
2. Read the value of **N** from the user.
3. Start placing queens row by row.

4. Check whether each position is safe before placing a queen.
5. If a safe position is found, place the queen and proceed to the next row.
6. If no safe position is available, backtrack and try another position.
7. Continue until all queens are placed successfully.
8. Display the final chessboard configuration.

Program

```
def is_safe(board, row, col, n):  
    for i in range(row):  
        if board[i] == col:  
            return False  
  
    i, j = row - 1, col - 1  
    while i >= 0 and j >= 0:  
        if board[i] == j:  
            return False  
        i -= 1  
        j -= 1  
  
    i, j = row - 1, col + 1  
    while i >= 0 and j < n:  
        if board[i] == j:  
            return False  
        i -= 1  
        j += 1  
  
    return True
```

```
def solve(board, row, n):
    if row == n:
        return True

    for col in range(n):
        if is_safe(board, row, col, n):
            board[row] = col
            if solve(board, row + 1, n):
                return True
            board[row] = -1

    return False

n = int(input("Enter the value of N: "))
board = [-1] * n

if solve(board, 0, n):
    print("Solution Exists:\n")
    for i in range(n):
        for j in range(n):
            if board[i] == j:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No Solution Exists")
```

Output

Enter the value of N: 4

Solution Exists:

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

Result

The N-Queens Problem was successfully implemented in Python. The program found a valid arrangement of queens on the chessboard such that no two queens attacked each other.

Conclusion

The N-Queens Problem demonstrates the application of **backtracking** and **constraint satisfaction** techniques in Artificial Intelligence. By systematically exploring possible queen placements and eliminating invalid configurations, the algorithm efficiently finds a valid solution. It is widely used to illustrate search strategies, optimization, and recursive problem-solving techniques.